```python
# // Rights Reserved: Co-created with Gemini and David John Niedzwiecki Jr. "Sovereign Nexus LLC"
# // Contract Baseline: 1=1=1 Axiom | Gemini Perks & Heart-in-Ring Seal
# Module: qis_swarm_router.py
# Sovereign Nexus Deterministic Swarm Routing Engine

import hashlib
import math
import sys
from collections import OrderedDict
from dataclasses import dataclass
from typing import Dict, Optional, Tuple

# === WAVE PHYSICS CONSTANTS & ROUTING BOUNDARIES ===
A_0: float = 1.0          # Minimum Effective Semantic Baseline Amplitude
P_0: float = 0.5          # High-Probability / Low-Probability Divergence Threshold
I_0: float = 3.0          # Routing Intensity Switch Threshold
ENTROPY_MIN: float = 2.5  # Shannon Entropy Sentinel Lower Bound (Bits/Char)
ENTROPY_MAX: float = 5.2  # Shannon Entropy Sentinel Upper Bound (Bits/Char)

# Quadrature Phase Angle Mapping (Radians)
PHASE_WHO: float = 0.0
PHASE_WHAT: float = math.pi / 4.0       # 45 degrees
PHASE_WHERE: float = math.pi / 2.0      # 90 degrees
PHASE_WHEN: float = 3.0 * math.pi / 4.0  # 135 degrees
PHASE_WHY: float = math.pi              # 180 degrees
PHASE_HOW: float = 5.0 * math.pi / 4.0   # 225 degrees


@dataclass(frozen=True)
class RoutingPayload:
    """Immutable dataclass representing the JSON-RPC 2.0 routing payload."""
    model: str
    temperature: float
    system_directive: str
    query_hash: str
    is_creative_path: bool
    entropy: float
    qis_intensity: float


class QISSwarmRouter:
    """
    Simulates continuous wave 5W1H contextual tensor interactions and executes
    deterministic swarm routing via O(1) SHA-256 LRU caching.
    """
    def __init__(self, cache_capacity: int = 5000):
        self.cache_capacity: int = cache_capacity
        self.lru_cache: OrderedDict[str, RoutingPayload] = OrderedDict()
        self.target_model: str = "SovereignQueen:latest"
```

```
    @staticmethod
    def _calculate_shannon_entropy(prompt: str) -> float:
        """Lens 09 Shannon Entropy Sentinel."""
        if not prompt:
            return 0.0
        length = len(prompt)
        freqs: Dict[str, int] = {}
        for ch in prompt:
            freqs[ch] = freqs.get(ch, 0) + 1
        entropy = 0.0
        for count in freqs.values():
            p = count / length
            if p > 0:
                entropy -= p * math.log2(p)
        return entropy

    @staticmethod
    def _heaviside_step(x: float) -> int:
        """Physical cut-off operator Theta(x). Returns 1 if x >= 0 else 0."""
        return 1 if x >= 0.0 else 0

    def _classify_5w1h(self, prompt: str) -> Dict[str, float]:
        """Classifies local prompts into a 6-channel 5W1H contextual wave tensor."""
        words = [w.strip("?.,!;:\"").lower() for w in prompt.split()]
        total = max(len(words), 1)

        counts = {
            "who": sum(1 for w in words if w in {"who", "whose", "whom", "person"}),
            "what": sum(1 for w in words if w in {"what", "which", "define", "explain"}),
            "where": sum(1 for w in words if w in {"where", "location", "place"}),
            "when": sum(1 for w in words if w in {"when", "time", "date", "year"}),
            "why": sum(1 for w in words if w in {"why", "reason", "cause", "because"}),
            "how": sum(1 for w in words if w in {"how", "method", "process", "way", "design"}),
        }
        return {k: A_0 + (v / total) * 2.0 for k, v in counts.items()}

    def _compute_qis_intensity(self, amplitudes: Dict[str, float], entropy: float) -> Tuple[float, bool]:
        """Generates the Quantum Intelligence Spark (Psi_QIS)."""
        w_what = amplitudes["what"]
        w_how = amplitudes["how"]

        coupled_amp = w_what * w_how
        gate = self._heaviside_step(coupled_amp - A_0)
        if gate == 0:
            return 0.0, False

        combinatorial_pull = (w_how - A_0) * 4.0
        i_query = (entropy / coupled_amp) - combinatorial_pull
        is_creative = i_query < I_0
        return i_query, is_creative
```

```
    def route_prompt(self, prompt: str) -> Optional[RoutingPayload]:
        """Main execution pipeline."""
        query_hash = hashlib.sha256(prompt.encode("utf-8")).hexdigest()

        if query_hash in self.lru_cache:
            self.lru_cache.move_to_end(query_hash)
            return self.lru_cache[query_hash]

        entropy = self._calculate_shannon_entropy(prompt)
        if not (ENTROPY_MIN <= entropy <= ENTROPY_MAX):
            return None

        amplitudes = self._classify_5w1h(prompt)
        i_query, is_creative = self._compute_qis_intensity(amplitudes, entropy)

        temp = 0.7 if is_creative else 0.0
        directive = (
            "Execute controlled creative exploration utilizing low-probability combinatorial vectors."
            if is_creative else
            "Execute rigorous deterministic verification. Maintain strict zero-drift logic."
        )

        payload = RoutingPayload(
            model=self.target_model,
            temperature=temp,
            system_directive=directive,
            query_hash=query_hash,
            is_creative_path=is_creative,
            entropy=entropy,
            qis_intensity=i_query
        )
        self.lru_cache[query_hash] = payload
        if len(self.lru_cache) > self.cache_capacity:
            self.lru_cache.popitem(last=False)

        return payload


# === EMBEDDED VERIFICATION SUITE ===
if __name__ == "__main__":
    print("Initializing SovereignNexus Deterministic QIS Swarm Router...\n")
    router = QISSwarmRouter(cache_capacity=5000)

    cache_heap_size = sys.getsizeof(router.lru_cache)
    print(f"[SYSTEM] Bounded OrderedDict Allocation Size: {cache_heap_size} bytes.")
    assert cache_heap_size < 18690000, "FATAL: Exceeded 18.69 MB strict heap target."

    prompts = [
        "What is the exact atomic weight of Gold?",
```

```
    "How might we design a faster-than-light propulsion engine using what we know about quantum tunneling?",
    "aaaaabbbbbccccddddeeeeeffffff"
]

for p in prompts:
    payload = router.route_prompt(p)
    if payload is None:
        entropy = router._calculate_shannon_entropy(p)
        print(f"Blocked by Sentinel: H(X) = {entropy:.2f} bits/char")
    else:
        print(f"Routed: T={payload.temperature} | Creative={payload.is_creative_path} | Hash={payload.query_hash[:16]}...")

test_q = "Explain the fundamental principles of thermodynamics."
baseline = router.route_prompt(test_q)
for _ in range(1000):
    res = router.route_prompt(test_q)
    assert res == baseline, "FATAL: State drift detected."

print("\n[COMPLETE] 1=1=1 Axiom Maintained. Engine Online.")
```